Office and PDF Tools

This document describes ragent's **office document tools** – the MS Office (`office_read`, `office_write`, `office_info`) and LibreOffice/OpenDocument (`libre_read`, `libre_write`, `libre_info`) families – plus the two PDF tools (`pdf_read`, `pdf_write`). It covers formats, JSON input schemas, worked examples, permissions, checkpointing, and configuration options.

All parsing and writing is done **in-process with pure-Rust crates**. There is no `soffice`/LibreOffice binary, no Microsoft Word, and no external converter to install: the "libre" tools read OpenDocument ZIP/XML directly, and the PDF tools use in-process extraction/writer crates. A workspace-wide grep for `soffice` returns zero matches.

Contents

1. Tool inventory
2. Enabling the tools
3. Format matrix
4. Tool reference
5. Output formats and truncation
6. Permissions and checkpointing
7. Implementation notes
8. Worked examples
9. Commands and configuration reference
10. Testing and sample files
11. Related documentation

---

1. Tool inventory

Eight tools are registered by `crates/ragent-tools-extended/src/lib.rs` (lines 430-437) in this order:

Tool name	Family	Purpose	Permission
`pdf_read`	PDF	Extract text or metadata from PDF files	`file:read`
`pdf_write`	PDF	Create new PDF documents	`file:write`
`office_read`	MS Office	Read Word/Excel/PowerPoint (OOXML) content	`file:read`
`office_write`	MS Office	Create Word/Excel/PowerPoint (OOXML) files	`file:write`
`office_info`	MS Office	Return JSON metadata for OOXML files	`file:read`
`libre_read`	LibreOffice	Read ODT/ODS/ODP content	`file:read`

Tool name	Family	Purpose	Permission
`libre_write`	LibreOffice	Create ODT/ODS/ODP files	`file:write`
`libre_info`	LibreOffice	Return structure summary for ODF files	`file:read`

All eight are implemented in `crates/ragent-tools-extended/src/` (modules `office_read.rs`, `office_write.rs`, `office_info.rs`, `libreoffice_read.rs`, `libreoffice_write.rs`, `libreoffice_info.rs`, `pdf_read.rs`, `pdf_write.rs`, plus shared helpers `office_common.rs` and `libreoffice_common.rs`).

---

2. Enabling the tools

The office/PDF family is **hidden from the LLM by default**. The `tool_visibility.office` switch (a bool, default `false`) controls whether the eight tools are advertised in `ToolRegistry::definitions()`; two consequences follow:

- When the switch is `false`, all tools in the family are suppressed from the tool definitions the model sees, so the model will not discover or call them on its own.
- Tools remain **registered and executable regardless of visibility** – a call by name still runs. Visibility only affects advertising, not executability.

Two ways to enable:

```
# 1. TUI slash command (persists to config)
/tools office on
```

```
// 2. ragent.json
{
  "tool_visibility": {
    "office": true
  }
}
```

Other `/tools` switches (`github`, `gitlab`, `teams`, `agents`, `plan`, `codeindex`, `masterfetch`, `browser`) behave the same way; `/tools office` without an argument reports the current state. The `/tools` handler persists the change with `Config::save()` (which invalidates the load cache, so the change is immediately visible to subsequent loads in the same process) and reports `[ok] 'office' visibility is now **true**`. A save failure is reported with a `[warn]` line and an `(unsaved)` status hint.

For reference, the default visibility switches are:

Switch	Default
`office`	`false` (hidden)
`github`	`false` (hidden)
`gitlab`	`false` (hidden)
`teams`	`false` (hidden)
`agents`	`false` (hidden)
`plan`	`false` (hidden)

Switch	Default
`codeindex`	`true`
`masterfetch`	`true`
`browser`	`true`
`finance`	`true`

---

3. Format matrix

Family	Formats supported	Legacy formats
`office_*`	`.docx` (Word), `.xlsx` (Excel), `.pptx` (PowerPoint) – OOXML only	`.doc`, `.xls`, `.ppt` are **rejected** with an actionable error
`libre_*`	`.odt` (text), `.ods` (spreadsheet), `.odp` (presentation) – OpenDocument	non-ODF extensions rejected
`pdf_*`	`.pdf`	–

Legacy binary Office formats (`.doc`, `.xls`, `.ppt`) are **rejected** with a message directing conversion: `Please convert to the modern OOXML format (.docx, .xlsx, .pptx)`. Files with no recognisable extension are rejected the same way. Convert legacy documents with LibreOffice, Word, or `soffice --convert-to docx` before using these tools.

---

4. Tool reference

4.1 `office_read`

Reads content from Word (`.docx`), Excel (`.xlsx`), or PowerPoint (`.pptx`) files.

Parameter	Type	Required	Notes
`path`	string	**yes**	Path to the file
`sheet`	string	no	Excel sheet name or index (0-based)
`range`	string	no	Excel cell range, e.g. `A1:D10` (1-based cells)
`slide`	integer	no	PowerPoint slide number (1-based)
`format`	enum	no	`text`, `markdown` (default), or `json`

The tool runs off the main thread (`spawn_blocking`), renders the selected portion, and truncates large output (Section 5). Markdown is the default presentation: paragraphs become text, tables become markdown tables, headings become `#`-prefixed lines.

4.2 `office_write`

Creates or overwrites OOXML files. Required: `path` + `content`. Optional `type` (`docx`, `xlsx`, or `pptx` – auto-detected from the file extension when omitted) and `title` (docx only). Parent directories are created automatically.

The `content` payload is format-specific JSON:

- **docx** – an array of blocks: `{"type": "paragraph" | "heading" | "bullet_list" | "code_block", "text": ..., "level": ..., "items": [...], "style": ...}` (only the fields relevant to the block type are required).
- **xlsx** – `{"sheets": [{"name": "Sheet1", "rows": [["a", "b"], ["1", "2"]]}]}`.
- **pptx** – `{"slides": [{"title": ..., "body": ..., "notes": ...}, ...]}`; a direct top-level array of slide objects is also accepted.

The writer tolerates several payload shapes for docx (3 variants) and pptx (5 variants) plus a top-level-key fallback, so slightly different JSON structures still succeed.

4.3 `office_info`

Takes only `path` and returns JSON metadata: title, author, creation date, page/slide/sheet counts, word count (docx), sheet names (xlsx), and slide titles (pptx). Legacy `.doc`/`.xls`/`.ppt` inputs are rejected.

4.4 `libre_read`

Reads `.odt`, `.ods`, and `.odp` content.

Parameter	Type	Required	Notes
`path`	string	**yes**	Path to the OpenDocument file
`sheet`	string	no	ODS only – sheet name or index
`range`	string	no	ODS only – cell range, e.g. `A1:D10`
`slide`	integer	no	ODP only – slide number (1-based)
`format`	enum	no	`text`, `markdown` (default), `json`

Implementation: ODS is read with the `calamine` crate; ODT and ODP are parsed by extracting text from the ODF ZIP's XML (via `quick-xml`).

4.5 `libre_write`

Creates OpenDocument files. Only `path` is required.

Parameter	Type	Notes
`path`	string	Destination file (`.ods`, `.odt`, or `.odp`)
`rows`	2-D string array	ODS content; header row first
`content`	array or text	ODT/ODP block array or plain text
`sheet_name`	string	ODS sheet name
`title`	string	Document title
`author`	string	Document author

Block payloads for ODT/ODP use `{"type": "paragraph" | "heading" | "bullet_list" | "ordered_list" | "code_block", ...}`; plain text is also accepted. ODS is written with the `spreadsheet-ods` crate; ODT/ODP are written as ODF ZIP + XML directly. Note the asymmetry with `office_write`: only `path` is required here.

4.6 `libre_info`

Requires `path`; the extension must be `.ods`, `.odt`, or `.odp`. Returns sheet list with dimensions (ODS), word/paragraph counts (ODT), or slide count (ODP).

4.7 `pdf_read`

Extracts text from PDF files in-process.

Parameter	Type	Required	Notes
`path`	string	**yes**	Path to the PDF
`start_page`	integer	no	1-based, inclusive
`end_page`	integer	no	1-based, inclusive
`format`	enum	no	`text`, `metadata`, or `json`

Page extraction works by extracting the whole document and filtering to the requested range, with an `lopdf` fast-path fallback. The metadata format returns path/format/line-count metadata; line counts reflect the (possibly truncated) text. Output is truncated at the shared 100 KB limit.

Encrypted PDFs are not supported: the vendored `pdf-extract` crate only decrypts through password-taking variants that `pdf_read` does not use, so an encrypted document errors out.

4.8 `pdf_write`

Creates a new PDF from a structured payload. Requires `path` + `content`; the `content` object has an optional `title` and an `elements` array whose items are `{type: "paragraph" | "heading" | "table" | "image", text, level, headers, rows, ...}` (`level` 1-3 for headings; `headers`/`rows` for tables). Layout is fixed A4 (210x297 mm) with 25 mm margins and built-in font sizes (11 pt body, 22 pt H1, 17 pt H2, 13 pt H3). Parent directories are created automatically.

---

5. Output formats and truncation

- Successful reads truncate at `MAX_OUTPUT_BYTES` = **100 KiB** (`office_common.rs`). The truncation notice explicitly tells the model to narrow the result using `sheet`/`range` (Excel, ODS) or `slide` (PowerPoint, ODP) instead of reading whole large documents.
- `pdf_read`'s reported line count reflects the text *after* truncation.
- Write errors embed a truncated JSON snippet of the offending payload in the error message.
- There is no offset/limit pagination parameter on the office tools (unlike MasterFetch's web fetches); pagination is by sheet, range, or slide only.

---

6. Permissions and checkpointing

- **Permission categories.** Read-side tools (`office_read`, `office_info`, `libre_read`, `libre_info`, `pdf_read`) require `file:read`; write-side tools (`office_write`, `libre_write`, `pdf_write`) require `file:write`. These flow through the standard configurable allow/deny/ask rule engine and the file-path guards – the same surface as the plain `read`/`write` tools. There are no office-specific permission rules.
- **Goal-loop write detection.** `LOOP_WRITE_TOOLS` (in `crates/ragent-agent/src/session/loop_state.rs`) includes `office_write`, `libre_write`, and `pdf_write`. Inside a `/loop` run, any call to these three tools counts as a destructive write: it arms the pre-loop snapshot/checkpoint machinery (forced checkpoints gated on `spec.checkpoints` and `checkpoint_timeout_secs`) and participates in the loop change summary/diffstat.
- **Auto-approval interplay.** `--yes` (alias `--no-prompt`) and YOLO mode auto-approve ordinary permission prompts, but forced loop checkpoints still fire for these write tools while loop checkpoints are enabled.
- **Visibility vs executability.** With `tool_visibility.office = false` the tools are hidden from the model but a call by name still executes.

---

7. Implementation notes

- All parsing and writing is pure Rust – no LibreOffice/soffice subprocess, no Word installation, no headless converters. The "libre" tools unpack the ODF ZIP and parse `content.xml`/`meta.xml` with `quick-xml`.
- Underlying crates (from `crates/ragent-tools-extended/Cargo.toml`): `docx-rust` and `ooxmlsdk` (Word), `calamine` (Excel/ODS reading), `spreadsheet-ods` (ODS writing), `zip`, `quick-xml`, `printpdf` (PDF writing), `pdf-extract` (PDF reading, vendored at `vendor/pdf-extract`), `lopdf` (PDF fast path).
- `pdf-extract` is a workspace-patched dependency: the root `Cargo.toml` declares `pdf-extract = "0.10"` and patches it to `vendor/pdf-extract`. Extraction is whole-document-in-memory (`extract_text_from_mem`); encrypted variants exist upstream but are unused.
- Both `office_read` and `office_write` execute on the tokio blocking pool (`spawn_blocking`), so large document work does not stall the async runtime.
- Office/PDF writes do not take their own file snapshots inside the tool; the snapshot/undo machinery that covers them is the session-level snapshot path and, inside goal loops, the forced-checkpoint machinery described in Section 6.

---

8. Worked examples

The examples below show the JSON arguments an agent passes to each tool. In the TUI you would normally just ask for the outcome ("read the table from report.xlsx"); the model issues these calls itself.

8.1 Read a Word document as markdown

```
{ "path": "docs/report.docx", "format": "markdown" }
```

8.2 Read one Excel sheet and range

```
{ "path": "data/metrics.xlsx", "sheet": "Q3", "range": "A1:D40", "format": "json" }
```

8.3 Read a single PowerPoint slide

```
{ "path": "decks/launch.pptx", "slide": 7 }
```

8.4 Create a Word document

```
{
  "path": "out/summary.docx",
  "type": "docx",
  "title": "Sprint Summary",
  "content": [
    { "type": "heading", "text": "Summary", "level": 1 },
    { "type": "paragraph", "text": "All planned work landed this iteration." },
    { "type": "bullet_list", "items": ["auth rewrite", "SSE hardening", "bench regressions fixed"
  ]
}
```

8.5 Create a spreadsheet

```
{
  "path": "out/budget.xlsx",
  "type": "xlsx",
  "content": {
    "sheets": [
      { "name": "Budget", "rows": [["Item", "Cost"], ["Hosting", "120"], ["CI", "45"]] }
    ]
  }
}
```

8.6 Create a presentation

```
{
  "path": "out/kickoff.pptx",
  "content": {
    "slides": [
      { "title": "Roadmap", "body": "Q3 objectives", "notes": "keep to 10 minutes" },
      { "title": "Risks", "body": "quota and supply", "notes": "" }
    ]
  }
}
```

8.7 Inspect OOXML metadata

```
{ "path": "docs/spec.pdf" }
```

(`office_info` on a `.docx` returns title/author/creation date/word count; on `.xlsx` it adds sheet names; on `.pptx` slide titles.)

8.8 Read OpenDocument files

```
{ "path": "sheets/totals.ods", "sheet": "Summary", "range": "A1:C12" }
```

```
{ "path": "notes/plan.odt", "format": "text" }
```

```
{ "path": "decks/pitch.odp", "slide": 3 }
```

8.9 Write an OpenDocument spreadsheet

```
{
  "path": "sheets/plan.ods",
  "sheet_name": "Plan",
  "rows": [["Phase", "Owner"], ["Discovery", "alice"], ["Build", "bob"]],
  "title": "Delivery plan",
  "author": "ragent"
}
```

8.10 Create a PDF report

```
{
  "path": "reports/audit.pdf",
  "content": {
    "title": "Dependency audit",
    "elements": [
      { "type": "heading", "level": 1, "text": "Summary" },
      { "type": "paragraph", "text": "All direct dependencies pass cargo-deny." },
      { "type": "table",
        "headers": ["Crate", "Licence", "Status"],
        "rows": [["pdf-extract", "MIT", "ok"], ["printpdf", "MIT", "ok"]] }
    ]
  }
}
```

8.11 Enable the family and drive a document task

```
# one-time enable (or use /tools office on in the TUI)
# ragent.json: { "tool_visibility": { "office": true } }
ragent run --agent general "read assets/pdf/Profile.pdf and summarise the candidate's experience"
```

Sample files for manual testing live in `assets/pdf/` (`Profile.pdf`, `vibeSDLC.pdf`) and `assets/officedocs/` (`testword1.docx`).

---

9. Commands and configuration reference

TUI

Command	Effect
`/tools`	Show the tools/visibility help

Command	Effect
`/tools office`	Report whether the office family is currently visible
`/tools office on` / `off`	Toggle visibility and persist to config
`/agents`	Agent diagnostics include the Office family status

Configuration (`ragent.json`)

Key	Type	Default	Effect
`tool_visibility.office`	bool	`false`	When `false`, all eight office/PDF tools are suppressed from `ToolRegistry::definitions()` (hidden from the LLM). Tools stay registered and executable regardless.

There are no office/pdf-specific keys beyond the visibility switch; all other behaviour (formats, payloads, limits) is fixed by the tool implementations.

Permissions

Category	Tools
`file:read`	`office_read`, `office_info`, `libre_read`, `libre_info`, `pdf_read`
`file:write`	`office_write`, `libre_write`, `pdf_write`

Configure allow/deny/ask rules for these categories in the `permissions` array of `ragent.json`, exactly as for the plain file tools.

---

10. Testing and sample files

- **Behavioural unit tests for the eight tools are currently absent** – there are no `test_office_*`, `test_libre_*`, or `test_pdf_*` tool-behaviour files under `crates/ragent-tools-extended/tests/`. Coverage that does exist is indirect: the visibility listing includes all eight names (`crates/ragent-agent/tests/test_t`
  the TUI `definitions()` check includes `office_read` (`crates/ragent-tui/tests/test_slash_commands.rs`),
  and the step-log input summary for `libre_read` is locked in by `crates/ragent-tui/tests/test_tool_display.rs`.
- Sample asset files (`assets/officedocs/testword1.docx`, `assets/pdf/Profile.pdf`, `assets/pdf/vibeSDLC.pdf`) are informal manual fixtures; nothing in `tests/` or `examples/` references them, so they are not CI coverage.
- `vendor/pdf-extract` is a workspace-patched dependency used by `pdf_read` (and MasterFetch's PDF path); it extracts text fully in memory with no external binary.

---

11. Related documentation

- `docs/howtos/reactagent.md` – the core per-turn agent loop that executes these tools.
- `docs/howtos/loopprogramming.md` – goal loops; relevant for how `office_write`/`libre_write`/`pdf_write` participate in forced checkpoints.
- `TUI-QUICKSTART.md` – quick reference for slash commands including `/tools`.
- `crates/ragent-tools-extended/DOCS.md` – curated descriptions of the extended tool families.
- `SPEC.md` – full configuration schema including `tool_visibility`.